MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión D del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_ajas_andre
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/ajas_andre/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas esa expresión.

Tu respuesta

```
el new es para poder crear una nueva cabeza en nuestro objeto
Resive una nueva seccion, ya sea int char float etc
```

Corrección. Reconoce que new sirve para crear algo del objeto, pero no menciona que ejecuta el constructor para inicializar la instancia. La variable no recibe 'una sección int, char o float': recibe la referencia al objeto recién creado.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué significa void en la cabecera de un método? Explica si ese método puede modificar un atributo.

Tu respuesta

```
Es parte de lo que nos regresa en la terminal al momento de hacer todo el proceso del nuevo
codigo
no lo puede modificar
```

Corrección. Incorrecta: void no tiene que ver con lo que se muestra en la terminal, y el método sí puede modificar un atributo. void indica que el método no devuelve ningún valor a quien lo llama, pero puede producir efectos como cambiar el estado del objeto o imprimir.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Para qué sirve this? Explica cómo distingue un atributo de un parámetro que tiene el mismo nombre.

Tu respuesta

```
Para poder referirse a la caracteristica especifica de un objeto, ya sea usuario u otro ubjeto igual, se puede diferenciar por medio del punto (.this )
```

Corrección. Acierta en que this sirve para referirse a una característica del objeto, pero no dice que es una referencia al objeto actual ni explica la distinción. En this.temperatura = temperatura, la izquierda es el atributo del objeto y la derecha es el parámetro del mismo nombre.

Pregunta 0.4 · Preguntas abiertas

Pregunta

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un int y cuando le pasas un arreglo?

Tu respuesta

```
una cadena de argumentos
```

Corrección. 'Una cadena de argumentos' no responde la pregunta. Con un int el método recibe una copia del número; con un arreglo recibe una copia de la referencia, así que puede modificar los elementos del arreglo compartido, pero no reasignar la variable de quien llama.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen JDK y JRE? Explica cuál necesitas para compilar un programa y por qué.

Tu respuesta

```
Sirve para el tipo de gusrdado que va a tener el archivo en java y como se va a leet con la computadra
```

Corrección. Incorrecta: JDK y JRE no son formas de guardar un archivo. El JRE es el entorno de ejecución (la JVM y sus bibliotecas); el JDK agrega herramientas de desarrollo como javac, y por eso se necesita para compilar.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer base = 5;
Integer copia = base;
int[] aumentos = {1, 2, 3};
for (int aumento : aumentos) {
    base = base + aumento;
```

```
    System.out.println(base / 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Tu respuesta

```
base=5
copia=5
6,7,8
3,3,4
J

3.0,3.0.4.0
```

Corrección. La salida correcta es 3 / 4 / 5 / D51 / 5.0. El primer 3 es correcto, aunque sobran las comas y las líneas extra 'base=5' y 'copia=5'. Después no acumula base (debía valer 6, 8 y 11), lo que da 3 y 4 en lugar de 4 y 5; 4.0 sigue de forma coherente esa traza y muestra el formato double, y 'J' no corresponde a la concatenación "D" + 5 + 1 = D51.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
if (true) { int voltaje = 12; }
System.out.println(voltaje);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
por que estamos definiendo un buleano con un if
```

Corrección. La clasificación 'No compila' es correcta. La causa no lo es: el problema no es definir un booleano, sino que voltaje se declara dentro del bloque del if y no existe fuera de él (ámbito).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
StringBuilder ruta = new StringBuilder("Q");
ruta.append(2 + 3);
System.out.println(ruta);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
solo proyecta la Q
```

Corrección. La clasificación 'Funciona' es correcta, pero la salida no: imprime Q5, porque primero se evalúa 2 + 3 = 5 y luego append lo añade a "Q".

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = null;
System.out.println(canales.length);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
solo proyectaria canales que es igual a null
```

Corrección. La clasificación 'Compila pero truena' es correcta, y está bien señalar que canales es null. Pero no imprimiría nada: al leer length de una referencia null el programa lanza NullPointerException antes de mostrar algo.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temperatura) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
static int ()


static  tempoeratura;
static fitja;
int[] aumentos = {1, 2, 3};
for (int aumento : aumentos) {
    base = base + aumento;
    System.out.println(/ 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Justificaciones

```
//se añade la class con el nombre de nuesto objeto,
se definen los nombre de nuestros atributos y de que seccion son, se denomina la temperatura y
elrango que va a tener los diferentes tipos de atributos para cada una de las funciones, se
dann las funciones del inicio y el fin de la temperatuira que puede tener
```

Corrección. Solo nombra los dos datos del termostato ('tempoeratura' y 'fitja') sin tipo y ambos como static, cuando temperatura debe ser de instancia y mutable, y etiqueta de instancia y final. Faltan la clase, el constructor con this, ajustar, la consulta y el reto: los extremos se declaraban como static final int MINIMO = 16 y MAXIMO = 28. El resto del campo es el código de R1 copiado y la justificación es genérica; solo nombra los atributos.

R4 · Código con repetición

Pregunta

Devuelve el promedio entero de los valores pares de pulsos usando for-each. Si no hay pares, devuelve 0. El arreglo puede estar vacío, no es null y admite negativos. La suma cabe en int. No modifiques el arreglo.

```
static int promedioPares(int[] pulsos)
```

Ejemplos (argumentos → retorno):

{3, 8, 2, 5} → 5 · {1, 3, 5} → 0 · {-4, 3, 6} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int promedioPares(int[] pulsos)
int [] promedio
{3, 8, 2, 5} { 1, 3, 5} {-4, 3, 6 };
promedio = promedio [] suam
promedio= promedio/2
  if (%promedio = 0){
  sitem.out.println(" es par :");
  }
   if else (%promedio<=> 0){ //diferente
   sitem.out.println(" no es numero par :");
   }
promedio= suma promedio;
sitem.out.println(" la suma es :"+promedio );
```

Corrección. La cabecera es correcta. No hay for-each ni recorrido del arreglo: escribe a mano los ejemplos, divide entre 2 en lugar de entre la cantidad de pares e imprime en vez de devolver. Bien la intención de usar % para detectar pares y de sumar. Lo correcto era recorrer con for-each, sumar y contar los valores con valor % 2 == 0, devolver 0 si la cuenta es 0 y si no, suma / cantidad.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la temperatura entera de una cámara de pruebas. Usa if / else-if / else y devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String estadoTermico(int temperatura)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 120	"INVALIDA"
0 a 29	"FRIA"
30 a 59	"TEMPLADA"
60 a 89	"CALIENTE"
90 a 120	"MUY CALIENTE"

Tu respuesta

```
static String estadoTermico(int temperatura){

if else (temperatura >= 0 && temperatura <= 29){
   Sistem.out.println("FRIA");
   }

if else (temperatura >= 30 && temperatura <= 59){
   Sistem.out.println("TEMPLADA");
```

```
   }

if else (temperatura >=60 && temperatura <= 89){
   Sistem.out.println("CALIENTE");
   }

if else (temperatura >= 90 && temperatura <= 120){
   Sistem.out.println("MUY CALIENTE");
   }
else (temperatura <0|| tempera>120  ){

   Sistem.out.println("INVALIDA ");
   }
}
```

Corrección. La cabecera es correcta y los rangos están bien: las cuatro categorías válidas, ambos extremos inválidos y las transiciones inclusivas 29/30, 59/60 y 89/90 (cabecera, flujo, lógica y límites completos). Imprime con System.out.println en lugar de devolver el texto con return, y hay errores de sintaxis: 'if else' en vez de 'else if', 'else' con condición y 'Sistem'.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.